

moving from a state named `state1` to a state named `state2`. It has the effect of applying a Fade effect to an object that's being added in the state:

- `<s:transitions>`
- `<s:Transition fromState="state1" toState="state2">`
- `<s:Fade target="{addedObject}"/>`
- `</s:Transition>`
- `</s:transitions>`

You also can use Parallel or Sequence effects to introduce more complex animation. The Parallel effect combines multiple effects that execute simultaneously. The following transition causes a Move effect and a Zoom effect to play simultaneously, creating the visual effect of an object “exploding” from the top-left corner of the application into its final position:

- `<s:Transition fromState="*" toState="detail">`
- `<s:Parallel target="{detailImage}">`
- `<s:Move xFrom="0" yFrom="0" xTo="300" yTo="50"/>`
- `<s:Zoom zoomHeightFrom="0" zoomWidthFrom="0"`
- `zoomHeightTo="1" zoomWidthTo="1"/>`
- `</s:Parallel>`
- `</s:Transition>`

As described earlier in this chapter, the Sequence effect enables you to play multiple effects consecutively. The following code declares a Transition with a Sequence that executes a horizontal Move effect first, and follows it with a vertical Move:

- `<s:Transition fromState="*" toState="detail">`
- `<s:Sequence target="{detailImage}">`
- `<s:Move xFrom="0" yFrom="0" xTo="300" yTo="0"/>`
- `<s:Move xFrom="300" yFrom="0" xTo="300" yTo="300"/>`
- `</s:Sequence>`
- `</s:Transition>`

The effects framework includes the following special classes that are designed to control when an object is added or removed from a presentation or when other actions are taken relative to when particular visual effects are executed in a transition,

- ▶ **AddAction** - Defines when an object is added to the presentation.
- ▶ **CallAction** - Calls a function of the defined target object.
- ▶ **RemoveAction** - Defines when an object is removed from the presentation.
- ▶ **SetAction** - Sets the value of a named property of an object. 